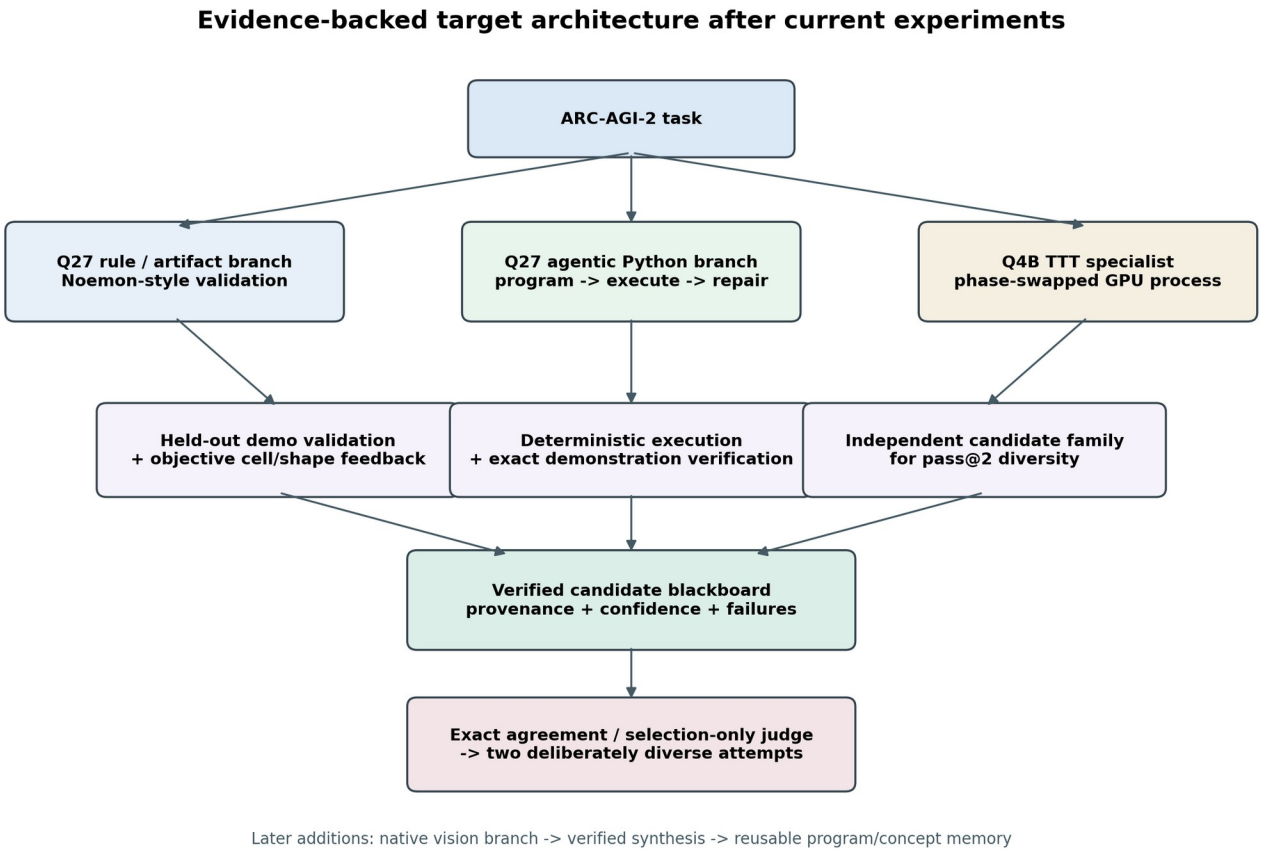
ARC-AGI-2 SOLVER RESEARCH DOSSIER

From a Qwen3 4B Test-Time-Training Baseline to a Verifier-Grounded Qwen3.8-27B Agentic Architecture

Experiment Record, Notebook and ZIP Audits, Failure Analysis, Research Synthesis, and Prioritized Roadmap



Research direction synthesized from the project experiments and external ARC-AGI-2 evidence.

Report date: 9 September 2026

Status: Working research report - Version 1.0

Primary compute target: Kaggle 4 x NVIDIA L4, no-internet evaluation

Central thesis

The project evidence no longer supports a strategy centered on scorer tuning or uninterrupted direct chain-of-thought. The strongest path is verifier-grounded, heterogeneous candidate generation: compact hypotheses or programs, objective execution/validation, bounded repair, independent candidate families, and pass@2-aware selection.

Contents

1. Executive Summary
2. Project Objective and Competition Constraints
3. Evidence Base and Artifact Corpus
4. Baseline 4B Solver: Architecture and Audit
5. Why Selection Tuning Cannot Reach the Strategic Goal
6. Dataset and EDA Findings
7. EXP-0: Distribution and Evaluation Harness
8. Qwen3.8-27B Runtime Feasibility
9. EXP-1: Direct Qwen Reasoning PROD_SMOKE
10. Noemon Source Audit and Gemini-to-Qwen Port
11. EXP-2A Static Preflight: Latest Result
12. External ARC-AGI-2 Research Synthesis
13. Integrated Research Insights
14. Target Architecture
15. Prioritized Next Experiments and Decision Gates
16. Risks, Reproducibility, and Research Discipline
17. Conclusion

Appendix A. Experiment Timeline

Appendix B. Artifact Ledger

Appendix C. Key Metrics Dashboard

References

Reading note

This dossier separates internal project evidence from external research evidence. Internal results are reproduced from the project notebooks and ZIP bundles. External claims are labeled by provenance: official/verified, peer-reviewed or preprint, repository self-report, or project research note.

1. Executive Summary

The project began with a strong but coverage-limited Qwen3 4B ARC solver whose public Kaggle leaderboard score varied roughly between 28 and 32.64. The strategic objective was not a marginal improvement; it was to identify architecture-level experiments with a plausible path toward 70+ accuracy. This distinction changed the research program. The project explicitly deprioritized small beam, scorer, and LoRA tuning unless those changes were necessary to support a larger architectural test.

The baseline audit produced the first decisive result: the frozen candidate pool contained the ground-truth output for only 55 of 172 public-evaluation query outputs. Under a hypothetical perfect selector that always picked the ground truth whenever it existed in the pool, the local equivalent score rose only from 33.50 to 41.17. This demonstrates that candidate generation coverage, not merely ranking quality, is the dominant bottleneck. Selection work remains useful, but it cannot by itself bridge the gap to the strategic target.

EXP-0 then established a frozen evaluation harness. Challenge-visible structural features separate the 120-task Evaluation split from the 240-task competition Test split with cross-fitted AUC 0.8882. The effective sample size of the propensity-weighted Evaluation set is only 32.57 tasks, so the weighted score is deliberately treated as exploratory. Raw Evaluation pass@1/pass@2 remains the primary factual benchmark, consistent with ARC Prize calibration guidance for the public, semi-private, and private evaluation distributions.

The Qwen3.8-27B-FP8 checkpoint was validated as a single tensor-parallel model across all four L4 GPUs. The engine successfully initialized at a 29,696-token context length using vLLM 0.19.0. However, EXP-1 showed that a direct reasoning interface is the wrong way to exploit this model under the current Kaggle budget. In the seven-task production-budget smoke test, every XHIGH request consumed the full 12,288-token generation budget, none reached the reasoning terminator, and none started a final answer. The non-thinking arm fared better operationally but still produced only one parse-valid task out of seven. Crucially, that one valid task (981571dc) produced a 30 x 30 grid with 896 of 900 cells correct. The model therefore exhibits meaningful latent ARC capability, but exactness, output control, and verification are major bottlenecks.

Research then shifted to Noemon, a public agentic ARC-AGI-2 solver built around a Reasoner -> Validator -> Refine loop, persistent thought history, self-consistency, and a judge. Rather than reimplementing the idea loosely, the project froze Noemon commit 54c7c3af9aa95a495586cba8caffc9ca526f9924 and audited its source. The audit found 121 files, 93 Python files, zero syntax errors, and 16/16 provider-independent tests passing. It also identified multiple compatibility traps that make a naive Gemini-to-Qwen provider rename unsafe: main-entry dispatch would bypass the Noemon batch path, schema dialects differ, Qwen historical reasoning has a different message representation, and local vLLM waves must replace Gemini remote batches without changing the state machine.

EXP-2A STATIC_PREFLIGHT was designed as a hard-gated, CPU-first proof of this exact port. It passed 51 of 52 hard gates. The sole failure is important: preserved Noemon reasoning history does not fit the currently validated 29,696-token local context plan for all Evaluation queries. Across 1,032 simulated stage-query combinations, 19 overflow; Reasoner Round 3 reaches a worst-case estimated context of 46,908 tokens. Therefore the current state is correctly classified as NO_GO_FIX_STATIC_PREFLIGHT. The next experiment is not a seven-task GPU smoke. The immediate priority is context remediation, followed by a real high-context Qwen gate.

Current research state

EXP-0 is complete and frozen. EXP-1 direct Qwen is complete and is a NO-GO for a full raw-direct run. The Noemon source

and compatibility surface are audited. EXP-2A static preflight is 51/52 hard gates passed but is currently blocked by context feasibility. The next step is EXP-2A1 Context Remediation, then EXP-2A2 QWEN_GATE, then a seven-task Noemon-Q27 smoke.

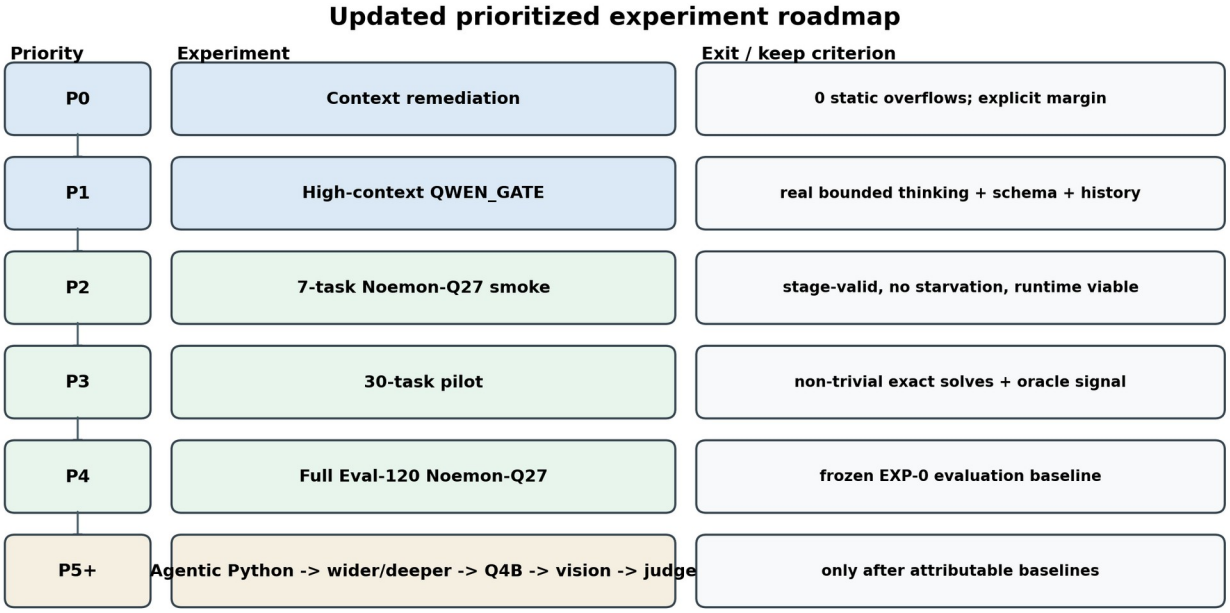


Figure 1. Updated priority order after the latest static preflight.

2. Project Objective and Competition Constraints

2.1 Strategic objective

The engineering objective is to maximize total exact ARC-AGI-2 solve rate under Kaggle constraints, with special emphasis on architecture changes that can plausibly lift the solver far beyond the existing approximately 28-32.64 public-LB range. The project target discussed throughout the research program is 70+, while the official ARC Prize 2026 bonus target is 85% on the private evaluation set [W1].

2.2 Scoring semantics

For each test input, the competition requires exactly two output grids. If either attempt matches the hidden ground truth exactly, the query scores 1; otherwise it scores 0. The final score is the average across task test outputs [W1]. This makes pass@2 diversity an architectural resource, not a formatting detail. Two correlated samples from the same hypothesis family may waste the second attempt, while two independently grounded candidate families can materially increase oracle coverage.

2.3 Deployment constraints

Constraint	Project implication
Kaggle notebook submission	All dependencies, models, source snapshots, and runtime artifacts must be available offline.
No internet during evaluation	No API-based Noemon/Gemini execution is possible in the competition run.
4 x NVIDIA L4 in the validated environment	Qwen3.8-27B must run tensor-parallel across all four GPUs; Q4B and Q27 cannot simply coexist as full active models.
Tight wall-clock budget	Search depth must be measured in solved tasks per hour, not only accuracy per request.
Exact-match scoring	99.56% cell accuracy still scores zero; verification and exact execution matter.
Two attempts	Candidate family diversity and selection must be optimized explicitly.

2.4 Evidence hierarchy

Tier	Examples	How used
A - Internal executed artifacts	Kaggle notebooks, ZIP bundles, exported JSON/CSV	Primary evidence for project conclusions.
B - Official ARC sources	Competition rules, official results, 2025 analysis	Used for benchmark constraints and verified scores.
C - Papers and open-source repos	Land, Agentic Coding, Noemon, NVARC, VARC, AB-MCTS	Used to design testable architecture hypotheses.
D - Public/self-reported systems	Noemon public 92.5; other community systems	Treated as directional evidence, not hidden-set proof.
E - Project research notes	Prior conversation research landscape	Used for hypothesis generation; important claims are re-verified where possible.

3. Evidence Base and Artifact Corpus

This report consolidates evidence from the complete project conversation archive, the current filesystem artifact set, executed Kaggle outputs, source-code audits, and a fresh verification pass over key external ARC-AGI-2 sources. The pasted conversation archive records the progression from the initial literature survey through EXP-0, Qwen direct reasoning, Noemon source freezing, audit, and EXP-2A preflight.

3.1 Primary internal artifacts reviewed

Artifact	Role	Status
arc-2026d-32-22lb-updated-v2 (20).ipynb	4B TTT base solver	Audited
31_experiment Notebook_arc-2026d-with-codex (22).ipynb	Candidate pool, scorer, oracle and failure analyses	Executed analysis inspected
arc-agi-2-with-qwen-3-8 (2).ipynb	Qwen3.8 runtime/throughput feasibility	Inspected
arc-2-advanced-eda (5).ipynb	Task structural EDA	Inspected
EXP0_ARC_AGI2_DISTRIBUTION_EVALUATION_BUNDLE.zip	Frozen distribution/evaluation harness outputs	Complete
EXP1_Q27_PROD_SMOKE_BUNDLE.zip	Production-budget direct Qwen smoke	Complete
NOEMON_ARC_AGI_SOURCE_SNAPSHOT.zip	Frozen Noemon source snapshot	Complete
NOEMON_SOURCE_AUDIT_BUNDLE.zip	File/function/provider/source audit	Complete
EXP2A_NOEMON_Q27_STATIC_PREFLIGHT_BUNDLE (1).zip	Latest exact-port static gate	Complete; NO-GO due context

3.2 Reproducibility discipline

A recurring project rule is that expensive Kaggle runs must produce machine-readable diagnostics and a single uploadable ZIP. The bundles preserve raw generations or request state, summary CSVs, JSON metadata, manifests, and hashes. This report extends that discipline by including an artifact ledger and SHA-256 hashes in Appendix B.

- Freeze external code before adaptation; record exact Git commit and hashes.
- Run compatibility/preflight checks before model allocation whenever possible.
- Treat silent fallback, implicit truncation, or schema coercion as correctness failures rather than conveniences.
- Separate architecture experiments so that gains remain attributable.
- Keep raw outputs and failure traces; they frequently reveal the next architecture bottleneck more clearly than aggregate score.

4. Baseline 4B Solver: Architecture and Audit

4.1 Baseline architecture

The base solver uses a Qwen3 4B ARC-specialized checkpoint with four independent GPU workers. Each L4 hosts its own 4B model process. At test time, each puzzle receives puzzle-specific LoRA test-time training, followed by augmented autoregressive search and candidate scoring.

```
ARC task
  -> per-puzzle LoRA TTT (r=256, rsLoRA, 1 epoch, lr=5e-5)
  -> 16-way train augmentation
  -> custom turbo_dfs / beam-like generation
  -> augmented NLL and beam scoring
  -> two selected candidate grids
```

Component	Observed implementation
GPU topology	4 separate L4 workers via multiprocessing; one 4B model replica per GPU.
TTT	LoRA r=256; q/k/v/o, gate/up/down, embeddings and lm_head; rsLoRA; 1 epoch.
Augmentation	Training augmentation n=16; evaluation augmentation approximately n=2.
Search	Custom recursive turbo_dfs; ARC token IDs; fixed beam/candidate thresholds.
Selection	Probmul/Kgmon-style ranking and dual-answer selection.

4.2 Audit findings

The audit identified several correctness and efficiency issues. These remain useful engineering work, but the project deliberately stopped treating them as the main route to 70+ because the candidate oracle experiment established a much lower structural ceiling.

- Potential mutable KV-cache contamination between recursive sibling branches in turbo_dfs; this was identified as a P0 risk but not conclusively proven.
- calc_scores sums token negative log-likelihood, creating an output-length bias.
- Generation is not grammar-constrained; invalid grids are rejected only after generation.
- Fixed max-score and beam constraints can prune useful candidates; adaptive escalation would be safer.
- The nominal 1600-second puzzle cap is not a hard recursive DFS deadline.
- Fallback zero-grid / NOT_PROCESSED handling can be counted as produced output, masking true processing failures.
- Sentinel scores such as 999 can distort candidate-local normalization.
- Full-vocabulary logits are repeatedly moved to CPU during recursive search, creating avoidable overhead.

5. Why Selection Tuning Cannot Reach the Strategic Goal

The most important baseline experiment freezes the candidate pool and asks a counterfactual: if the selector were perfect and always chose the ground truth whenever it appeared anywhere in the generated pool, how high could the current system score?

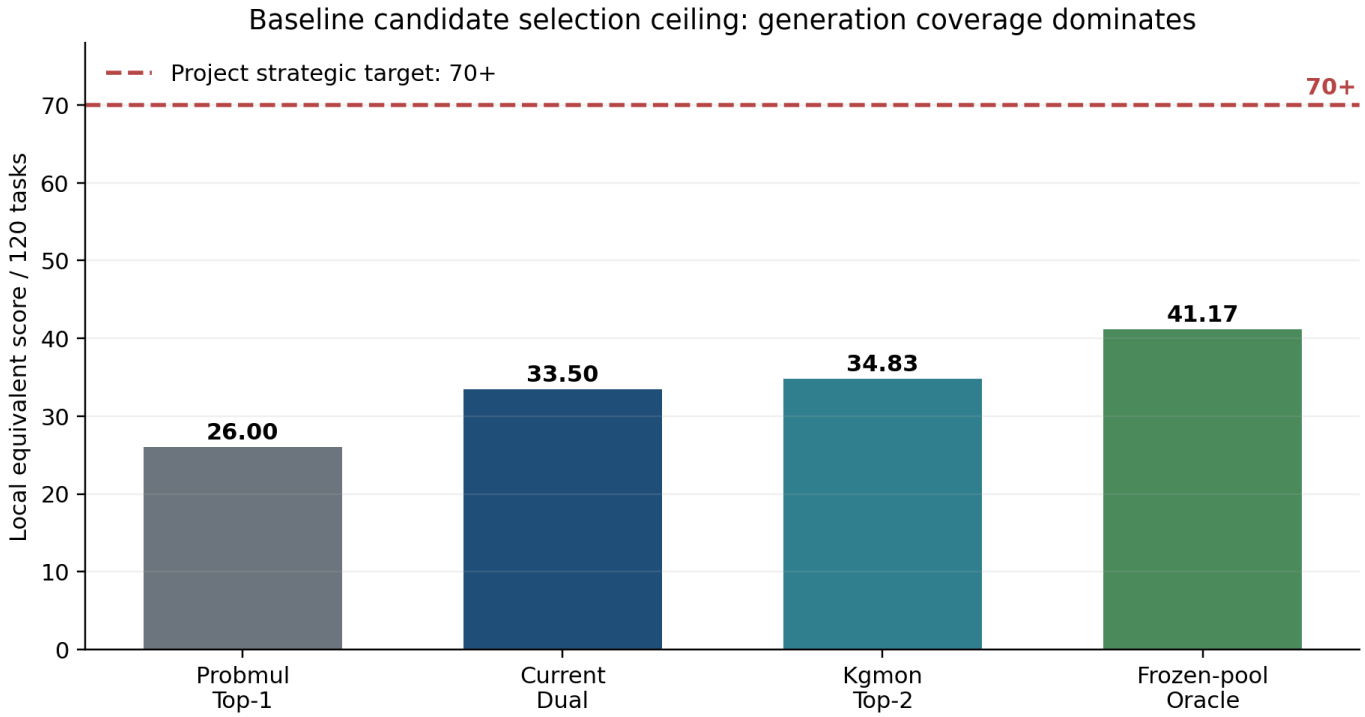


Figure 2. Candidate-pool ceiling versus current selection policies. The 70+ line is a project goal, not a measured score.

Metric	Result
Probmul Top-1	26.00
Probmul Top-2	33.50
Kgmon Top-2	34.83
Current dual	33.50
Ground truth present in pool	55 / 172 query outputs (32.0%)
Frozen-pool oracle	41.17 / 120 equivalent
Oracle gain over current dual	+7.67

The rank distribution adds nuance: among cases where the ground truth existed in the pool, 37 were rank 1, 10 rank 2, 4 rank 3, 3 rank 5, and 1 rank 9 under the best observed ranking. Better selection could recover some missed rank-3-to-rank-5 solutions, but even an oracle selector cannot approach 70 while the candidate pool itself covers only 32% of query outputs.

Strategic implication

Scorer and beam improvements are secondary. The architecture must increase candidate generation coverage and exactness. A selector cannot recover a candidate that was never generated.

6. Dataset and EDA Findings

6.1 Task structure

The project EDA highlights a heterogeneous task distribution and several failure modes for naive one-rule or fixed-budget solvers. The competition Test split contains 240 visible tasks and 259 test queries. Most tasks have one query, but 17 tasks are multi-query. The number of demonstration pairs ranges from 2 to 8, with 132 of 240 Test tasks having exactly three demonstrations.

Test n_train	Task count
2	45
3	132
4	43
5	14
6	4
7	1
8	1

Output expansion can be extreme: demonstration outputs can reach 900 cells, and many examples expand output area by more than 10x relative to input. This is one reason direct token-by-token grid emission is expensive and why executable transformations are attractive: a short program can describe a very large output deterministically.

6.2 Exact demonstration fit is not sufficient

A diagnostic simple-rule study produced a central lesson. Some hand-coded transforms fit all visible demonstrations yet generalize poorly to held-out targets. In the study summarized during the project, 72 training tasks were demo-fitted by simplistic rules; among non-skipped targets, only about 28.6% were actually correct. Therefore "fits every shown demonstration" must not be equated with "has inferred the intended abstraction."

Research rule

Whenever possible, use out-of-sample demonstration validation, independent application, or executable verification. Exact fit to the same examples that generated a hypothesis is necessary but not sufficient.

7. EXP-0: Distribution and Evaluation Harness

7.1 Purpose

EXP-0 created a solver-independent evaluation harness before more expensive architecture experiments. It computes label-free structural task features, quantifies visible Evaluation/Test covariate shift, builds cross-fitted propensity weights, and defines exact pass@1/pass@2 scoring semantics. It intentionally does not load an LLM.

7.2 Results

Metric	Result
Training tasks	1,000
Evaluation tasks / queries	120 / 172
Test tasks / visible queries	240 / 259
Eval-vs-Test cross-fitted AUC	0.888229
Cross-fitted log loss	0.424510
Weight min / median / max	0.2285 / 0.2285 / 5.7127
Effective sample size	32.57 of 120 Evaluation tasks

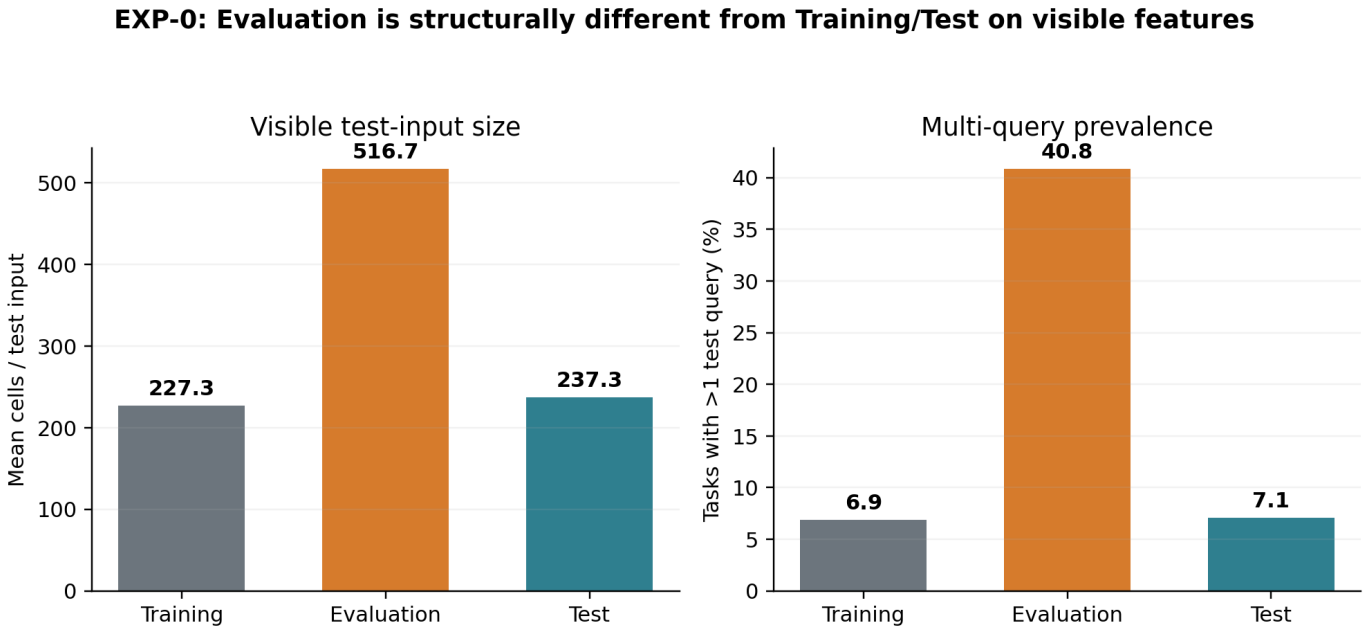


Figure 3. Examples of challenge-visible structural shift measured by EXP-0.

The Evaluation split is visibly larger and much more multi-query than Training or Test. However, the official ARC framing treats the public, semi-private, and private evaluation sets as calibrated for score comparability. Consequently, the project does not use Eval/Test visible feature separability as evidence that one split is intrinsically harder or that a hidden score should shift in a particular direction.

7.3 Frozen evaluation policy

- Primary: raw Evaluation pass@1 and pass@2.
- Secondary: category breakdowns, invalid/missing outputs, attempt diversity, and candidate complementarity.
- Exploratory only: Test-relevance-weighted Evaluation and propensity-balance diagnostics.

- Do not select an experiment winner from weighted score alone; the weighting effective sample size is only 32.57.
- Do not use handcrafted EDA difficulty labels as solver ground truth.

8. Qwen3.8-27B Runtime Feasibility

8.1 Validated checkpoint and topology

The secondary model is the Qwen3.8-27B-FP8 repacked checkpoint at the Kaggle model path used in the project. It reports architecture Qwen3_5ForConditionalGeneration, model type qwen3_5, and dynamic E4M3 FP8 quantization. Unlike the 4B baseline, it runs as one tensor-parallel model distributed across all four L4 GPUs.

Runtime item	Validated value
GPU	4 x NVIDIA L4, approximately 22.03 GiB each
Model	Qwen3.8-27B-FP8 repacked
Architecture	Qwen3_5ForConditionalGeneration / qwen3_5
vLLM / Torch / CUDA	0.19.0 / 2.10.0+cu128 / CUDA 12.8
Tensor parallelism	TP=4; one model across all GPUs
Validated context	29,696 tokens in EXP-1 smoke
max_num_batched_tokens	32,768
max_num_seqs	16
gpu_memory_utilization	0.90
Compatibility	enforce_eager=True; custom all-reduce disabled; prefix caching off

8.2 Throughput envelope

Earlier forced-generation benchmarks showed that batch size 16 was the throughput winner at several completion lengths. Approximate measured rates were 800 tasks/hour at 256 tokens, 309 tasks/hour at 1,024, 168 tasks/hour at 2,048, and 87.6 tasks/hour at 4,096. These tests used forced generation with EOS ignored, so they should be interpreted as throughput characterizations rather than end-to-end task latency predictions.

8.3 Output length pressure

Known direct Evaluation target JSONs have a median of about 974 tokens, P95 about 3,070.5, and maximum about 3,608. A 4,096-token final-answer reserve can therefore represent all known direct outputs in the current serialization. The problem exposed by EXP-1 was not simply that final grids are too long; it was that reasoning and output were coupled in one uninterrupted generation.

9. EXP-1: Direct Qwen Reasoning PROD_SMOKE

9.1 Design

EXP-1 was originally intended as a controlled direct-capability map: native non-thinking inference versus Qwen XHIGH reasoning, using the same tasks and output contract. After an initial short smoke demonstrated infrastructure but not capability, a corrected production-budget smoke used seven deterministic stress tasks with the same completion ceilings planned for a full run.

Task	Stress role
981571dc	Longest prompt
13e47133	Largest target JSON + large test input
1ae2feb7	Multi-query stress
8698868d	Two-demo ambiguity + median prompt
21897d95	Dynamic shape
e8686506	Shortest prompt
195c6913	Many colors

9.2 Infrastructure passed

All 15 code cells completed without Python exceptions. The preload self-test passed template, parser, unfinished-thinking, multi-query, invalid-rejection, tail-loop, propensity, active-task, and filesystem checks. The model loaded once at 29,696 context, and the full notebook completed in approximately 52.2 minutes.

9.3 Direct XHIGH failed as an interface

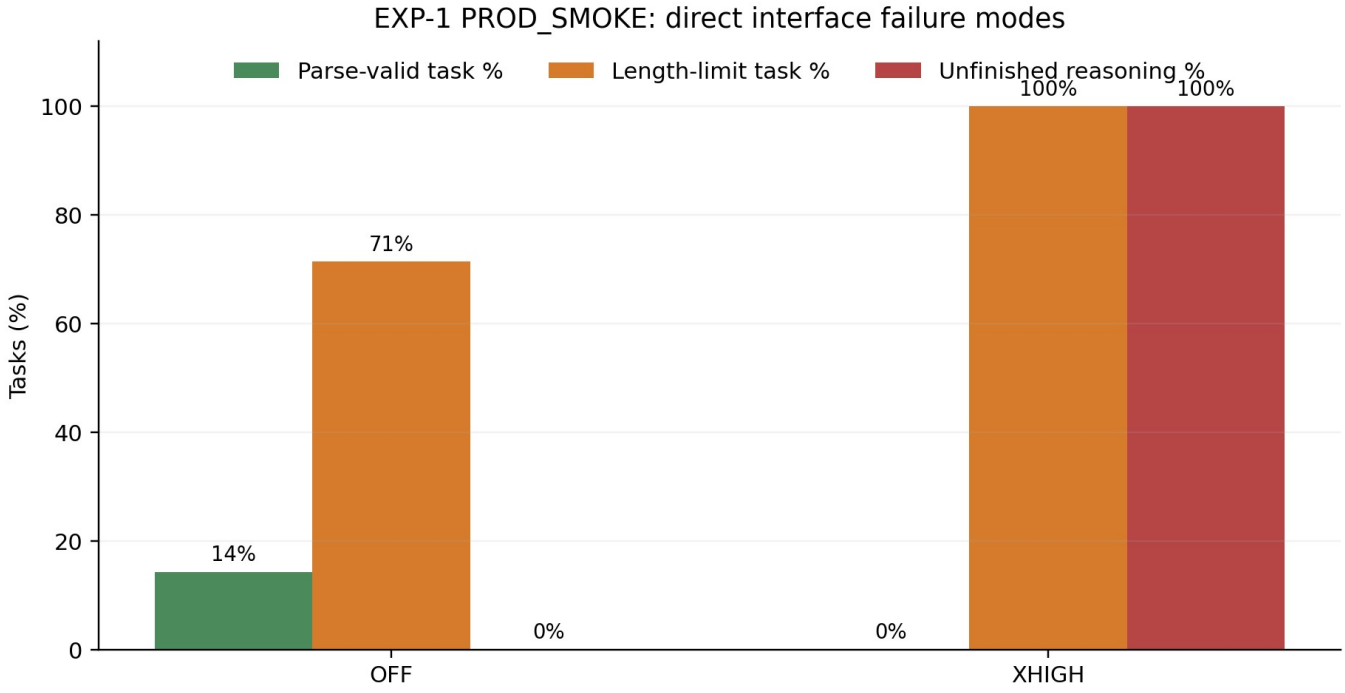


Figure 4. EXP-1 task-level validity and truncation diagnostics.

Metric	OFF	XHIGH
Successful engine task records	7 / 7	7 / 7
Parse-valid tasks	1 / 7 (14.29%)	0 / 7

Length-limit tasks	5 / 7 (71.43%)	7 / 7 (100%)
Reasoning end seen	100%	0%
Unfinished reasoning	0%	100%
Average generated tokens	3,195.7	12,288
Batch wall time	678.2 s	1,963.1 s

Every XHIGH request used exactly 12,288 generated tokens, hit the length limit, failed to reach the end of the thinking block, and never started a final answer. Tail-loop detection remained zero, so the primary failure was not a trivial repeated-token loop. The model continued deliberating until the budget ended. This falsifies the current free-form XHIGH-direct-grid design under the available runtime envelope.

The OFF arm also revealed an interface mismatch. Most completions ignored the "final JSON only" instruction, produced long prose analyses, and then truncated. Structured generation is therefore not an optional polish for future stages; it is a compatibility requirement.

9.4 The 981571dc near-miss

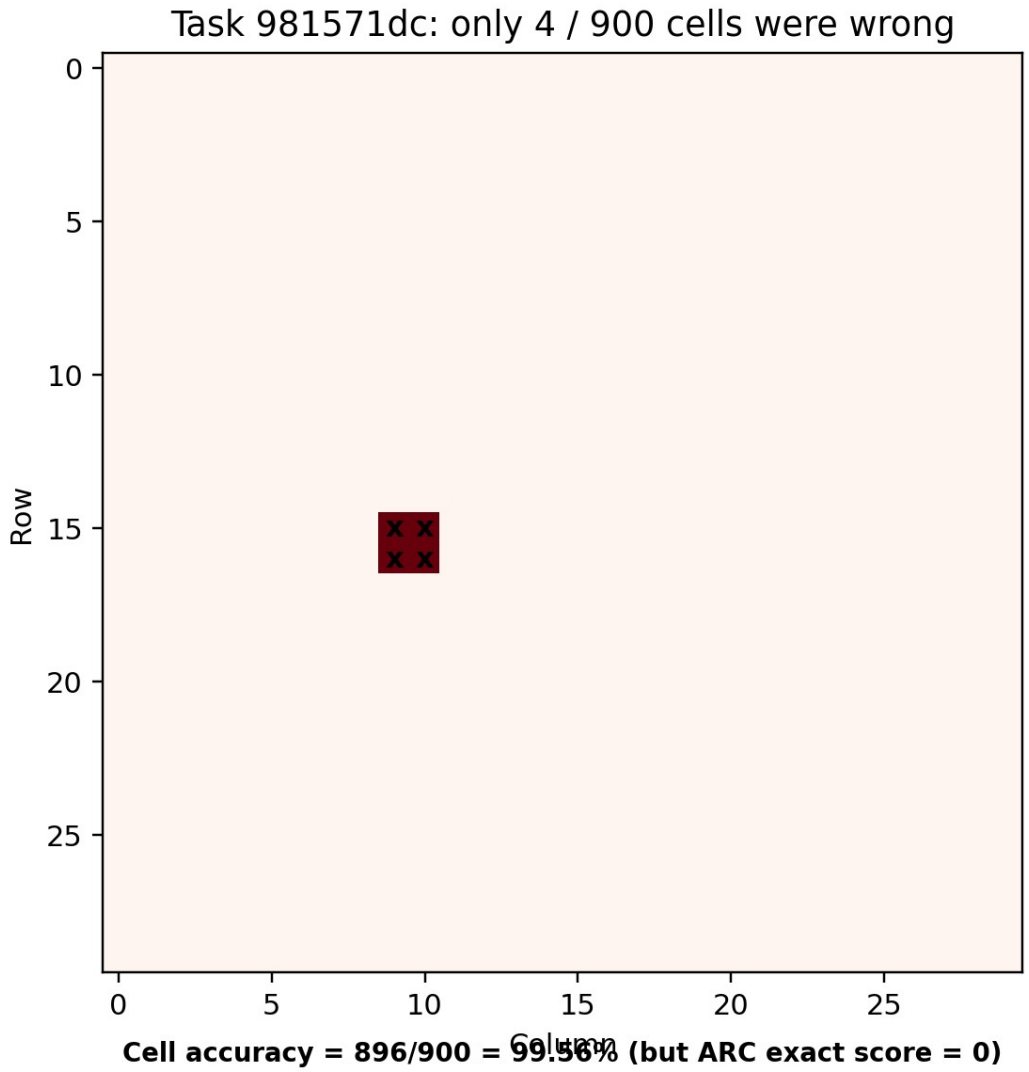


Figure 5. Error locations in the only strict-valid OFF output. Four wrong cells made an otherwise 99.56% cell-accurate grid score zero.

Task 981571dc is the strongest evidence that Qwen3.8-27B should not be discarded. The OFF arm emitted a strict-valid 30 x 30 output in 1,807 generated tokens. It matched 896 of 900 cells; only four cells were wrong. Under ARC exact scoring, the result is still zero. The failure therefore points directly toward objective verification, correction, and executable or artifact-based reasoning rather than simply more raw thought tokens.

9.5 EXP-1 decision

NO-GO for the old full run

Do not spend 5-10 GPU hours on the old 120-task raw XHIGH/OFF direct design. The infrastructure is viable; the interface is not. The next architecture must separate hypothesis formation, validation, and exact output production.

10. Noemon Source Audit and Gemini-to-Qwen Port

10.1 Why Noemon became the next baseline architecture

Noemon is especially relevant because its source separates reasoning from application. A Reasoner produces natural-language transformation instructions; a distinct Validator tests those instructions; failed validation produces concrete feedback; successful instructions are applied to the test input; repeated candidate agreement can terminate early; otherwise a judge selects among previously generated candidates. The repository reports 92.5% on its public evaluation rerun at about $3.9/task [W4]. The ARC community leaderboard currently labels the same system as "ARC-AGI-2 Public Train" at 92.5% [W3], so this report treats the result as a self-reported public result, not semi-private or private verification.

```
Reasoner Round 1
  -> instruction artifact
  -> held-out Validator
  -> objective failure feedback
  -> Reasoner Round 2 / Round 3
  -> test candidate + test validation
  -> independent final solvers
  -> exact duplicate self-consistency
  -> selection-only Judge if needed
```

10.2 Frozen source snapshot

Audit item	Result
Repository	noemon/noemon-arc-agi
Frozen commit	54c7c3af9aa95a495586cba8caffc9ca526f9924
Commit date	2026-04-13
Files / directories	121 / 26
Python files	93
Python syntax errors	0
Provider-independent tests	16 / 16 passed

10.3 Compatibility traps found before GPU implementation

Trap	Why a naive port is unsafe	Port rule
Gemini import/registry coupling	Installing/importing the wrong adapter path can require Google SDKs and fail offline.	Isolate the local Qwen provider boundary.
main.py dispatch	Only the Noemon Gemini batch provider is routed into Noemon batch semantics.	Use a dedicated local Noemon-Qwen runner; do not rely on generic provider

		dispatch.
Schema dialect	Noemon fallback schema uses Gemini-style types/keys; vLLM expects standard JSON Schema.	Recursively translate schema syntax while preserving field semantics.
History representation	Qwen expects reasoning_content + content for preserved thinking.	Store a tagged Noemon envelope and normalize to Qwen-native history at rendering.
Remote batch vs local vLLM	Gemini job submission/polling is not available offline.	Preserve wave/state-machine semantics but execute all ready requests in a local vLLM wave.
Reasoning-level mismatch	Gemini high/medium is not identical to Qwen xhigh/medium/low.	Map explicitly and prove actual template/render/runtime behavior.
Context accumulation	Persistent R1->R2->R3 thought history can be much larger than a single direct prompt.	Simulate every stage for every Evaluation query before GPU allocation.

10.4 Qwen-specific runtime contract

Qwen3.8 officially supports reasoning_effort values xhigh, medium, and low, and enables preserved thinking by default [W10]. vLLM 0.19.0 exposes both thinking_token_budget and structured_outputs in SamplingParams [W11]. These are necessary but not sufficient facts: the exact local checkpoint and vLLM wheel must still prove reasoning termination, structured JSON generation, their joint compatibility, and history round-tripping on the actual 4xL4 environment.

11. EXP-2A Static Preflight: Latest Result

11.1 Preflight philosophy

EXP-2A STATIC_PREFLIGHT was designed to fail cheaply. It does not load the 27B model. Instead, it verifies frozen source integrity, original Noemon semantics, ARC data conversion, leakage protections, JSON-schema conversion, Qwen model/config/template contracts, vLLM wheel symbols, history serialization, state-machine mock paths, checkpoint behavior, and an all-Evaluation context-budget simulation.

11.2 Gate outcome

Gate summary	Result
Hard gates	52
Passed	51
Failed	1
Status	NO_GO_FIX_STATIC_PREFLIGHT
Sole hard failure	budget_plan_v0_all_eval_contexts_fit
Simulated stage-query rows	1,032
Overflows	19
Worst estimated context	46,908 vs validated 29,696

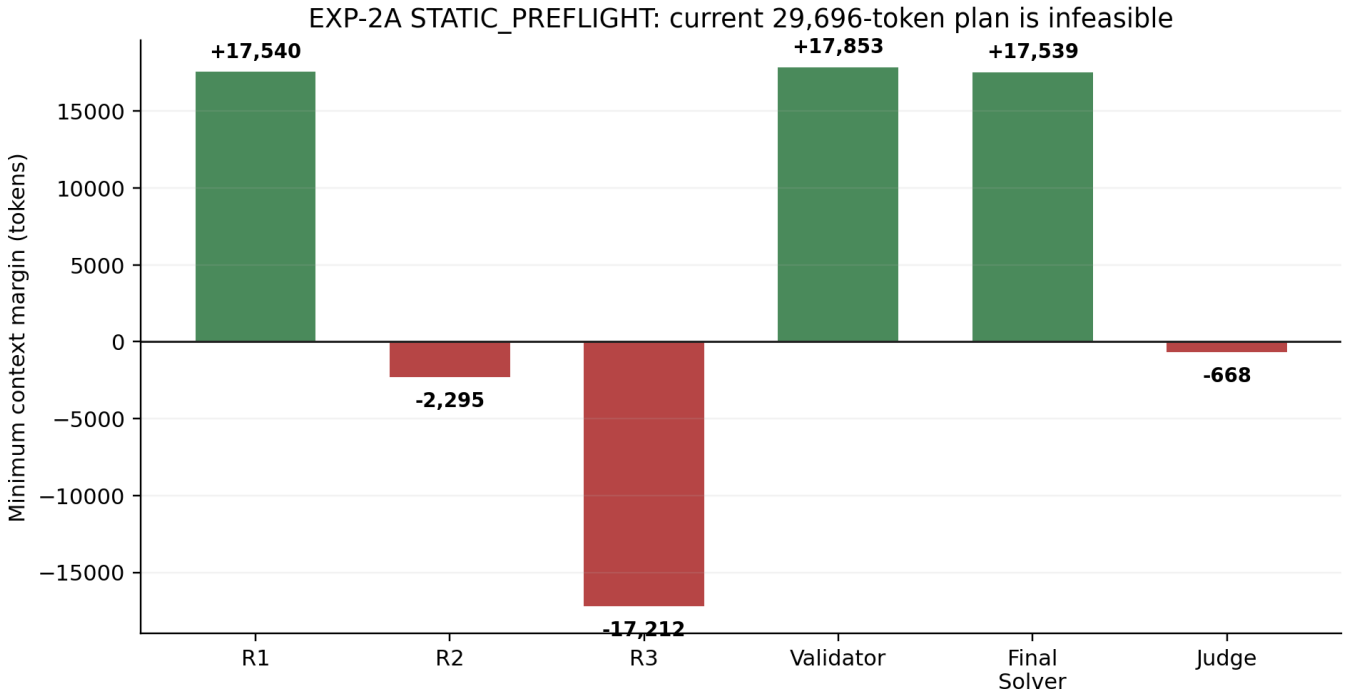


Figure 6. Minimum context margin by Noemon stage under the current 29,696-token engine plan.

Stage	Max total context	Min margin	Overflows
Reasoner R1	12,156	+17,540	0
Reasoner R2	31,991	-2,295	3
Reasoner R3	46,908	-17,212	15
Validator	11,843	+17,853	0
Final solver	12,157	+17,539	0
Judge	30,364	-668	1

11.3 Why this failure is valuable

The single failure prevented an expensive and misleading next run. Without this simulation, the local Noemon-Q27 implementation could have silently truncated preserved reasoning history, failed only on large tasks late in the run, or appeared to work on a seven-task subset while violating the intended architecture on the full Evaluation distribution. The preflight therefore succeeded as a research instrument even though its final status is NO-GO.

11.4 Current budget plan

Stage	Qwen effort	Thinking budget	Answer reserve
Reasoner R1	xhigh	3,072	2,048
Reasoner R2	xhigh	3,072	2,048
Reasoner R3	xhigh	3,072	4,096
Validator	medium	1,024	4,096
Final solver	xhigh	2,048	4,096
Judge	xhigh	1,024	256

The worst offender is task 981571dc in Reasoner Round 3 at 46,908 estimated total context, followed by several other large or multi-query tasks. A 49,152-token engine context would leave only about 2,244 tokens of margin at the worst case; 53,248 would leave about 6,340. Because the official Qwen model itself

supports much longer native context, the immediate uncertainty is local GPU KV-cache/runtime feasibility, not a model-architecture limit.

Immediate next step

Do not run the Noemon seven-task GPU smoke yet. First produce a context-remediation plan with zero static overflows, then run a real high-context Qwen micro-gate on 4xL4.

12. External ARC-AGI-2 Research Synthesis

The project deliberately re-read the recent ARC literature and open-source systems after each major failure. The important pattern is not that every high-scoring system uses the same implementation; rather, independent lines of work repeatedly converge on external verification, diverse candidate generation, execution, refinement grounded in evidence, and selective use of expensive reasoning.

12.1 High-value systems and lessons

Work / system	Evidence status	Key result	Lesson for this project
Noemon	Repo self-report; public provenance caveat	92.5% public result claimed	Compact rule artifact + held-out validation + bounded refinement + self-consistency.
Johan Land, Modality-Driven Search	72.9% semi-private reported in paper; code released	72.9% semi-private, 76.1% public	Independent text/image/code hypotheses; holistic judge can recover minority-correct candidates; prescriptive refinement can reduce diversity.
Agentic Coding	Controlled public-eval experiment	GPT-OSS 120B 6.11->26.38; GPT-5.2 59.81->73.36 in reported setting	Stateful Python execution can expose capability that uninterrupted CoT does not.
Sakana Multi-LLM AB-MCTS	Open implementation / research	Adaptive tree search on ARC-AGI-2	Decide dynamically between WIDER fresh branches and DEEPER evidence-grounded refinement.
ARC Prize 2025 / NVARC	Official 2025 competition result	Top private score ~24%	Synthetic data + ARChitects-style Qwen 4B + TRM; model/TTT branch remains complementary.
VARC	Open vision system; private result reproduction described	Vision-first ARC formulation	A visually grounded candidate family can be genuinely complementary to text/program reasoning.

12.2 Noemon and Land are not contradictory

Land reports a negative result for prescriptive prompting and naive iterative refinement, while Noemon reports strong performance with a refine loop. The combined interpretation is more precise than either slogan. Repeated self-critique on one anchored hypothesis can collapse diversity. Refinement becomes more defensible when it is bounded and driven by objective external failure evidence. When a branch fails broadly or stops improving, the search should widen to a fresh hypothesis rather than endlessly polishing the same idea.

12.3 Why Agentic Coding matters especially for Qwen3.8

EXP-1 showed the local Qwen model spending 12,288 tokens reasoning without reaching an answer. Agentic Coding provides an independent mechanism for breaking that failure mode: reason briefly, act by executing code, inspect objective evidence, then reason again. A program also compresses large grid outputs: the model can emit a short transformation function, while deterministic execution produces the exact large grid.

12.4 Research landscape in one sentence

Convergence pattern

Strong 2025-2026 ARC work increasingly treats inference as search over verifiable artifacts rather than a single prompt-to-grid mapping: generate hypotheses/programs, execute or validate them, branch when needed, preserve useful state, and use a judge only after candidate quality exists.

13. Integrated Research Insights

Insight	Evidence	Consequence
Generation coverage dominates selection	Only 55/172 ground truths in frozen 4B pool; oracle 41.17	Invest first in new hypothesis families and verification, not scorer micro-tuning.
Direct deep reasoning is not automatically useful	XHIGH 7/7 maxed 12,288 with no final answer	Bound reasoning and interleave with validation/action.
Q27 latent ability is real	981571dc: 896/900 correct cells	Do not discard Q27; move it to artifact/program roles where exactness can be checked.
Exact demo fit can overfit	Simple-rule diagnostic generalizes poorly	Use held-out demo validation and independent application.
Output contracts matter	OFF mostly produced prose despite JSON-only instruction	Use structured outputs at validator/final/judge boundaries.
Persistent history is a double-edged feature	Noemon requires history; static port overflows 29.7K	Treat context capacity as an architecture constraint and hard-gate it.
Judge value depends on oracle	Selector cannot recover missing candidates	Delay expensive judge/synthesis until candidate oracle rises.
Pass@2 should encode diversity	Competition awards either exact attempt	Make attempt_2 a deliberately different verified candidate family.

13.1 Negative knowledge: what has been ruled out

- Running the old 120-task free-form XHIGH direct Qwen baseline.
- Expecting scorer/beam tuning alone to lift the current 4B architecture to 70+.
- Treating demonstration-perfect rules as trusted abstractions without out-of-sample checks.
- Treating EXP-0 Test-weighted Evaluation as an official or primary score.
- Assuming Noemon 92.5 is semi-private/private verified.
- Silently truncating preserved reasoning history to make Noemon fit the current context.
- Loading Q27 and four Q4B replicas simultaneously; the validated configurations each consume the full GPU set.
- Adding a powerful judge before candidate generation/oracle quality improves.

14. Target Architecture

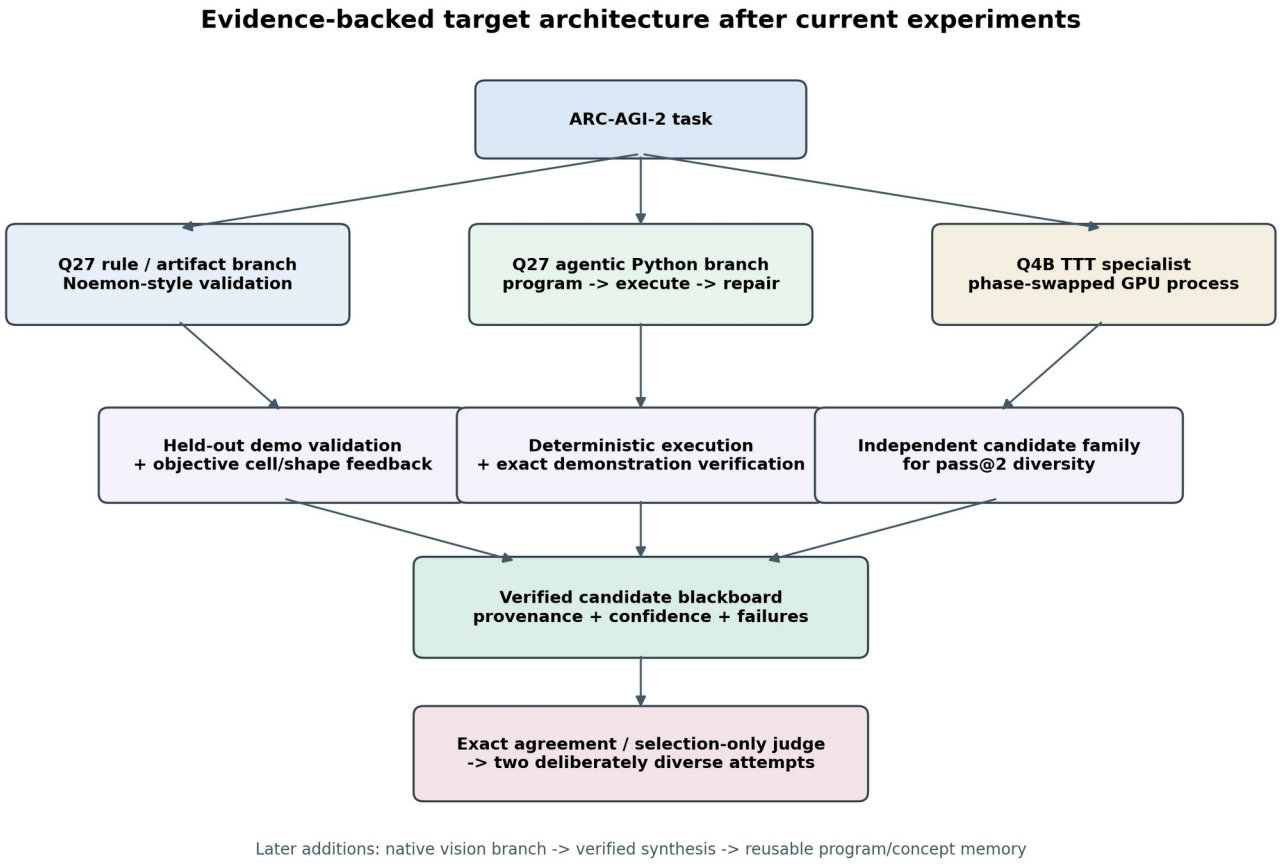


Figure 7. Long-term architecture synthesized from project evidence. The immediate experiments test only the Q27 Noemon branch; the other branches are added later for attribution.

14.1 Immediate branch: exact Noemon-Q27 compatibility port

The first architecture baseline must keep Noemon decision logic and prompts as stable as possible while replacing only the Gemini-specific provider/runtime boundary. Core source files remain hash-locked. The local runner collects all ready Noemon BatchRequestSpec objects, renders Qwen messages, executes one local vLLM wave with per-request sampling/schema settings, persists manifests/checkpoints, and advances the original Noemon state machine.

14.2 Second branch: stateful executable program synthesis

```
Q27 hypothesis
  -> Python solve(grid) program
  -> execute on every demonstration
  -> exact mismatch / shape feedback
  -> bounded branch-local repair
  -> verified test execution
```

This branch is intentionally distinct from the natural-language Noemon artifact branch. The strongest signal will be the union oracle: how many queries are solved by either branch, and how often independent artifacts converge on the same exact output.

14.3 Third branch: existing Q4B TTT specialist

The 4B solver should be retained as a complementary transduction specialist rather than the main reasoning architecture. Since Q27 TP4 and four Q4B replicas both occupy all GPUs, integration should use phase-swapped processes: run Q27, exit and free GPU memory, run Q4B, exit, optionally reload Q27 for judging, then merge candidates.

14.4 Later branches

- Native vision / VARC-inspired representation to recover spatial hypotheses missed by language/program branches.
- Holistic selection-only judge after the candidate oracle is high enough to justify selection compute.
- Verified synthesis only if synthesis produces a new program/artifact that is executed and revalidated before use.
- Reusable primitive/program/concept memory inspired by evolutionary program synthesis, SOAR, ArcMemo, and NVARC-style synthetic learning.

15. Prioritized Next Experiments and Decision Gates

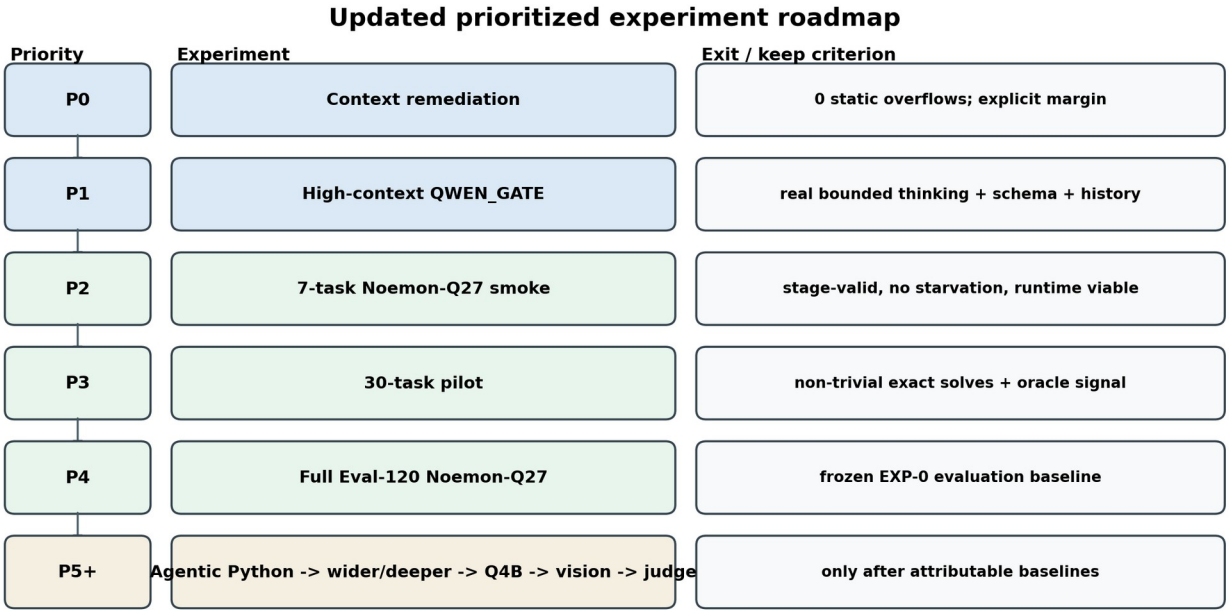


Figure 8. Research priority sequence. Each expensive step is gated by a measurable exit criterion.

P0 - EXP-2A1 Context Remediation Preflight

This is the immediate next experiment. It remains CPU/static and compares explicit context policies. The preferred first hypothesis is to preserve the original Noemon thought-history semantics and raise the local model context to approximately 53,248 tokens, because the current worst estimated stage is 46,908 tokens. Controlled fallback policies should also be simulated in case high-context local inference is not viable: reduced thinking budgets and compact semantic history. These fallbacks must be labeled as architecture deviations, not silently substituted.

- Keep: zero static overflows across all 1,032 simulated stage-query rows; explicit safety margin; no hidden truncation.
- Kill: any plan that relies on dropping history without recording the semantic change.

P1 - EXP-2A2 High-Context QWEN_GATE

Load the exact Qwen3.8-27B checkpoint once on 4xL4 and test the smallest possible real requests before any task smoke. If the context-remediation plan selects approximately 53K, this gate first asks whether the engine can initialize with that length without OOM or catastrophic throughput loss.

- ReasoningConfig accepted by the exact vLLM 0.19.0 wheel and checkpoint.
- thinking_token_budget forces a real reasoning end on a tiny request.
- StructuredOutputsParams(json=...) works on the exact model.
- Thinking budget and structured outputs work together.
- One local wave accepts mixed per-request SamplingParams.
- A real Reasoner R1 -> R2 history roundtrip preserves reasoning exactly once.

Kill criteria: OOM, inability to force reasoning termination, schema incompatibility, corrupted history, or an obviously infeasible throughput envelope.

P2 - EXP-2A3 Noemon-Q27 7-task Smoke

Run the original Noemon control flow on seven diverse tasks. Record every stage request/response, validation outcome, candidate, duplicate decision, judge use, prompt/generation tokens, and wall time. The purpose is architectural validity and runtime characterization, not a representative accuracy estimate.

- Keep: stage-validity approximately 90%+; no systematic token starvation; projected full runtime below roughly 11 hours with safety margin; at least some meaningful exact or near-exact signal.
- Kill/modify: repeated reasoning starvation, widespread schema failure, or projected runtime outside the Kaggle envelope.

P3 - 30-task Noemon-Q27 Pilot

A 30-task pilot should precede a full 120-task run unless the seven-task smoke is unexpectedly decisive. It provides a much better estimate of exact solve rate, validation pass rate, branch behavior, and tasks/hour without committing the full budget.

- Primary metrics: raw exact pass@1/pass@2, candidate oracle, held-out validator exact-pass rate, calls/task, tokens/stage, runtime/task.
- Decision: proceed to full only if solve signal is non-trivial and runtime remains feasible.

P4 - Noemon-Q27 Full Evaluation-120

This becomes the architecture-level Q27 baseline, scored with the frozen EXP-0 evaluator. It answers a clean scientific question: how much of Noemon transfers to a local open-weight Qwen3.8-27B model when only provider/runtime compatibility is changed?

P5 - Agentic Python Branch

Add a stateful Python REPL/executor as a separate candidate family. Compare Noemon-only, code-only, union oracle, and selected pass@2. This is the highest-priority architecture addition after a stable Noemon baseline because external research and the project near-miss both indicate that objective execution can unlock latent model capability.

P6 - Cross-Validation and Adaptive WIDER/DEEPER

For tasks with at least three demonstrations, use held-out demonstrations to falsify hypotheses. For two-demo tasks, prioritize independent multi-start because leave-one-out validation becomes underdetermined. Add a small AB-MCTS-inspired controller only after branch quality is measurable: deepen a promising branch when objective feedback is local and improving; widen when failure is broad, structural, or stagnant.

P7 - Q4B TTT Integration

Phase-swap the existing 4B transduction specialist into the candidate blackboard. Measure complementarity rather than assuming ensemble value. The key number is union oracle gain relative to the Q27 branches.

P8 - Native Vision Branch

Add a visually grounded candidate family after text/program branches are stable. Vision is high-priority research, but not before the cheaper and better-verified executable branch is measured.

P9 - Holistic Judge and Verified Synthesis

Use an expensive judge only when oracle analysis shows a meaningful selection gap. Start selection-only: the judge must choose an existing candidate, not hallucinate a replacement. If synthesis is later allowed, the synthesized artifact must be executable/verifiable before it can become an attempt.

P10 - Reusable Memory, Primitive Library, and Synthetic Data

Longer-term work should capture successful programs, abstractions, and repair traces, then reuse them across tasks. This is where Eric Pang-style program libraries, SOAR self-improvement, ArcMemo-like concept memory, NVARC synthetic generation, and a compact ARC primitive library become relevant.

16. Risks, Reproducibility, and Research Discipline

16.1 Technical risks

Risk	Impact	Mitigation
Context/OOM	Port fails only on hard tasks or late waves	All-120 static context simulation; high-context micro-gate first.
History corruption	Noemon refinement no longer reproduces intended semantics	Tagged history envelope + exact render fingerprints + roundtrip tests.
Structured-output incompatibility	Prose/malformed grids or reasoning constrained incorrectly	Schema converter unit tests + real combined reasoning/schema microtest.
Runtime explosion	Architecturally good solver misses Kaggle deadline	Wave batching, per-stage telemetry, 7-task and 30-task projections, kill gate.
Candidate correlation	Pass@2 wasted	Track provenance; later use distinct artifact/code/Q4B/vision families.
Validation overfitting	Rule fits visible demos but not hidden target	Held-out demo validation; fresh independent branch when evidence is weak.
Selector illusion	Judge looks strong while oracle is low	Measure candidate oracle before investing in judge sophistication.
Fallback masking	Invalid work counted as output	No zero-grid success fallbacks; explicit missing/invalid state.

16.2 Reproducibility requirements for every expensive run

- Exact notebook hash and active profile.
- Model path, config, tokenizer/chat-template fingerprint, vLLM/Torch/CUDA versions.
- Raw generations or program artifacts plus parsed outputs.
- Per-stage prompt and generation token counts.
- Finish reasons, truncation/invalid flags, wall time, and batch/wave IDs.
- Exact pass@1/pass@2 plus candidate oracle and complementarity.
- Checkpoint/resume state for long runs.
- One ZIP bundle containing manifest and SHA-256 hashes.

17. Conclusion

The project has already achieved its most important early objective: it has turned a broad "improve the ARC score" problem into a sequence of falsifiable architecture questions. The 4B candidate oracle rules out selector-only optimism. EXP-0 prevents misleading validation. EXP-1 rules out free-form direct XHIGH as the current Qwen interface while simultaneously revealing strong latent near-exact capability. The Noemon audit converts a high-scoring public idea into a precise source-level porting problem. EXP-2A then catches a full-Evaluation context failure before a costly GPU run.

The evidence now favors a verifier-grounded search architecture. Qwen3.8-27B should reason into compact artifacts or executable programs, not be asked to think indefinitely and then manually paint a large exact grid. Hypotheses should be tested against held-out demonstrations or deterministic execution; failures should produce objective feedback; refinement should be bounded; unsuccessful branches should widen; and pass@2 should deliberately preserve independent candidate families.

The immediate research priority is narrow and concrete: solve the Noemon context-feasibility problem without silently changing semantics, prove the resulting local Qwen runtime contract, and only then measure a seven-task and 30-task port. If that branch produces meaningful candidate coverage, the next major jump is a stateful Python program branch. Q4B TTT, vision, and holistic judging are retained as later complementary layers rather than discarded.

Highest-value experiment now

EXP-2A1 Context Remediation -> EXP-2A2 High-Context QWEN_GATE. Success means zero static context overflows followed by a real 4xL4 micro-run that proves bounded reasoning termination, structured output, mixed-stage batching, and preserved R1->R2 history on the exact Qwen3.8 checkpoint.

Appendix A. Experiment Timeline

Phase	Experiment / work	Outcome
Baseline	4B TTT solver + candidate/scorer audit	Public LB historically ~28-32.64; local dual 33.50.
Baseline analysis	Frozen candidate oracle	55/172 GT present; oracle 41.17 -> generation coverage bottleneck.
EDA	Advanced structural EDA	Large variability, multi-query, expansion, n_train heterogeneity; exact demo fit is unsafe.
EXP-0	Distribution/evaluation harness	Complete and frozen; AUC 0.8882; primary metric remains raw Eval pass@1/pass@2.
Q27 feasibility	TP4 runtime/throughput notebook	Qwen3.8-27B FP8 viable on 4xL4; vLLM 0.19.0.
EXP-1	Direct OFF vs XHIGH production smoke	NO-GO for old full direct run; XHIGH answer starvation; OFF format failure; one 99.56% near-miss.
Research pivot	Noemon + Land + Agentic Coding + AB-MCTS synthesis	Verifier-grounded artifact search selected as main direction.
Source engineering	Noemon snapshot/audit	Commit frozen; 16/16 provider-independent tests; compatibility traps mapped.
EXP-2A	STATIC_PREFLIGHT	51/52 hard gates pass; NO-GO due 19 context overflows.
NEXT	EXP-2A1 / EXP-2A2	Context remediation, then real high-context Qwen gate.

Appendix B. Artifact Ledger

ID	Artifact	SHA-256 prefix	Role
A1	arc-2026d-32-22lb-updated-v2 (20).ipynb	e563cbd93c32	Base 4B solver
A2	31_experiment Notebook_arc-2026d-with-codex (22).ipynb	69a64354b6b1	Candidate/oracle audit
A3	arc-agi-2-with-qwen-3-8 (2).ipynb	7d91965196ec	Q27 throughput/runtime
A4	arc-2-advanced-eda (5).ipynb	1a9384d4b2e0	EDA
A5	EXP0_ARC_AGI2_DISTRIBUTION_EVALUATION_HARNESS.ipynb	5774fd9783de	Canonical EXP-0 notebook
A6	EXP0_ARC_AGI2_DISTRIBUTION_EVALUATION_BUNDLE.zip	cdf2ccab014b	Canonical EXP-0 results
A7	EXP1_Q27_7TASK_PROD_SMOKE_CORRECTED_FINAL.ipynb	7e53e2acd835	EXP-1 smoke notebook
A8	EXP1_Q27_PROD_SMOKE_BUNDLE.zip	c47095736e44	EXP-1 outputs
A9	NOEMON_ARC_AGI_SOURCE_SNAPSHOT.zip	55e215ac54ad	Frozen Noemon source
A10	NOEMON_SOURCE_AUDIT_BUNDLE.zip	30be3e321437	Noemon source audit
A11	EXP2A_NOEMON_Q27_STATIC_PREFLIGHT.ipynb	2f31bf7a5a42	EXP-2A preflight notebook
A12	EXP2A_NOEMON_Q27_STATIC_PREFLIGHT_BUNDLE (1).zip	ef93cac266b9	Latest EXP-2A results
A13	Pasted markdown(20260909-043615).md	3b861ff4bff1	Conversation/research archive

Hash prefixes are included for compactness. Full SHA-256 values were computed from the files available to this report and should be retained in the project manifest if the dossier is later used as a formal reproducibility record.

Appendix C. Key Metrics Dashboard

Area	Metric	Value
4B baseline	Current dual local score	33.50
4B baseline	Frozen-pool oracle	41.17
4B baseline	GT present in pool	55/172 (32.0%)
EXP-0	Eval-vs-Test AUC	0.888229
EXP-0	Weight ESS	32.57/120
Q27 runtime	Validated context	29,696
EXP-1 XHIGH	Unfinished reasoning	100%
EXP-1 XHIGH	Parse-valid	0%
EXP-1 OFF	Parse-valid	14.29%
EXP-1	981571dc cell accuracy	896/900 = 99.56%; exact score 0
Noemon audit	Provider-independent tests	16/16
EXP-2A	Hard gates passed	51/52
EXP-2A	Static context overflows	19/1,032 stage-query rows
EXP-2A	Worst estimated context	46,908
Current state	GPU smoke permission	NO - context remediation first

References

Web sources were re-checked on 9 September 2026. Project artifacts refer to Appendix B.

[W1] ARC Prize 2026 - ARC-AGI-2 Competition. https://arcprize.org/competitions/2026/arc-agi-2

[W2] ARC Prize 2025 Results & Analysis - Year of the Refinement Loop. https://arcprize.org/blog/arc-prize-2025-results-analysis

[W3] ARC Prize Community Leaderboard. https://arcprize.org/leaderboard/community

[W4] Noemon ARC-AGI-2 solver repository. https://github.com/noemon/noemon-arc-agi

[W5] Johan Land - Modality-Driven Search with Holistic Trace Judging for ARC-AGI-2. https://arxiv.org/abs/2606.31543

[W6] Sakana AI - Multi-LLM AB-MCTS for ARC-AGI-2. https://github.com/SakanaAI/ab-mcts-arc2

[W7] Agentic coding improves ARC AGI 2 performance across models. https://pivotools.github.io/pivotools-quarto-blog/posts/agentic_coding_arc_agi/

[W8] NVARC solution to ARC-AGI-2 2025. https://github.com/1ytic/NVARC

[W9] VARC - ARC Is a Vision Problem. https://github.com/lillian039/VARC

[W10] Qwen3.8-27B-FP8 official model card. https://huggingface.co/Qwen/Qwen3.8-27B-FP8

[W11] vLLM 0.19.0 SamplingParams - thinking_token_budget and structured_outputs. https://docs.vllm.ai/en/v0.19.0/api/vllm/sampling_params/

[W12] vLLM 0.19.0 Structured Outputs configuration. https://docs.vllm.ai/en/v0.19.0/api/vllm/config/structured_outputs/

[W13] ARC Prize 2025 competition details. https://arcprize.org/competitions/2025

Project artifact citations used in the report

A1-A13 are internal project artifacts listed in Appendix B. The report also relies on machine-readable outputs inside A6, A8, A10, and A12: run metadata, scoring summaries, generation diagnostics, source audit indices, gate results, context-budget summaries, and source integrity manifests.